Operators and Decorators Quiz Practice Problems

Note 1. There are three ways to attach behavior to a class in python: you can hand-write the dunder method that python calls, you can use a class decorator to generate the dunder method for you, or you can inherit the method from a parent class. In this notes packet we study all three, and we show how the built-in decorators `@dataclass` and `@total_ordering` are just class decorators that generate dunders.

1 Class Hierarchies

Note 2. When a class `B` derives from a class `A`, we write `class B(A):`. The function `super()` returns the next class in the *method resolution order* (MRO), which for simple cases is just the parent class `A`. Because the parent class owns its own state, its `__init__` method is responsible for setting that state; the child's `__init__` should therefore call `super().__init__(...)` to run it. If the child forgets this call, then any attributes that only the parent sets will not exist on the child.

Problem 3. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class A:
    pass
class B(A):
    def __init__(self, x):
        self.x = x
try:
    b = B(5)
    print('b.x=', b.x)
except AttributeError:
    print('AttributeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 4. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class A:
    def __init__(self, xs):
        self.xs = xs
class B(A):
    def __init__(self, x):
        self.x = x
try:
    b = B(5)
    print('b.xs=', b.xs)
except AttributeError:
    print('AttributeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

1.1 Exceptions

Note 5. Exceptions are just classes. The hierarchy of the exceptions you are required to know is:

```
BaseException
`-- Exception
    |-- LookupError
    |   |-- IndexError
    |   `-- KeyError
    |-- RuntimeError
    |   `-- RecursionError
    |-- AttributeError
    |   `-- FrozenInstanceError
    |-- TypeError
    |-- ValueError
    |-- AssertionError
    `-- ZeroDivisionError
```

`LookupError` is the parent of `IndexError` and `KeyError`, so an `except LookupError` block catches both list out-of-bounds and dict missing-key errors. `FrozenInstanceError` is a kind of `AttributeError`, raised when assigning to a field of a `@dataclass(frozen=True)` instance.

Problem 6. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def foo(xs):
    return foo(xs)
try:
    foo([])
except RuntimeError:
    print('RuntimeError')
except RecursionError:
    print('RecursionError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $12 \;/\; 16 = 0.75$

Problem 7. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
try:
    xs = [1, 2, 3]
    print('xs[10]=', xs[10])
except LookupError:
    print('LookupError')
try:
    d = {'a': 1}
    print('d[b]=', d['b'])
except LookupError:
    print('LookupError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

2 Decorators

Note 8. A *decorator* is a function that takes a function and returns a function. For example, we saw in a previous lab that `lru_cache` is a decorator that *memoizes* recursive functions to make them more efficient. The syntax `@decorate def f(x): ...` is "syntactic sugar" for `f = decorate(f)`. Decorators can modify functions, classes, and any other object type in python.

Problem 9. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def memoize(f, cache={}):
    def g(n):
        if n not in cache:
            cache[n] = f(n)
        return cache[n]
    return g

@memoize
def fib(n):
    if n < 2:
        return n
    return fib(n-1) + fib(n-2)
print('fib(20)=', fib(20))
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 10. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def memoize(f, cache={}):
    def g(n):
        if n not in cache:
            cache[n] = f(n)
        return cache[n]
    return g

@memoize
def f(n):
    return n * n

@memoize
def g(n):
    return n + 1

print('f(5)=', f(5))
print('g(5)=', g(5))
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

Note 11. The `functools.lru_cache` decorator is a library implementation of memoization. It uses the arguments to the decorated function as the dictionary keys, so the arguments must be *hashable*. Mutable containers like lists and dicts are not hashable, and passing them to an `lru_cache`'d function raises `TypeError`.

Problem 12. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import functools
@functools.lru_cache
def f(xs):
    return sum(xs)
try:
    print('f([1, 2, 3])=', f([1, 2, 3]))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 13. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import functools
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
@functools.lru_cache
def f(v):
    return v.x + v.y
try:
    print('f(Vec(1, 2))=', f(Vec(1, 2)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 14. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import functools
@functools.lru_cache
def f(n):
    return n * n
f(3)
f(3)
f(4)
info = f.cache_info()
print('hits=', info.hits)
print('misses=', info.misses)
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

3 Dunder Zoo

Note 15. The `__repr__` method is used to display objects in the REPL and inside containers. The `__str__` method is used by `print`. The default `__str__` just calls `__repr__`.

Problem 16. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __repr__(self):
        return 'R'
    def __str__(self):
        return 'S'
v = Vec(1, 2)
print(v)
print([v])
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Note 17. The default `__eq__` for objects is identity: two distinct objects are never equal. Python also gives every object a default `__hash__` based on its identity. There is a contract between `__eq__` and `__hash__`: if two objects are equal, they must have the same hash. When you override `__eq__`, python automatically sets `__hash__ = None`, which makes the class unhashable. This is to prevent the class from silently violating the contract.

Problem 18. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
a = Vec(1, 2)
b = Vec(1, 2)
print('a == b:', a == b)
print('a == a:', a == a)
print('hash(a) == hash(b):', hash(a) == hash(b))
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 19. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
a = Vec(1, 2)
b = Vec(1, 2)
print('a == b:', a == b)
try:
    print('hash(a):', hash(a))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 20. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
    def __hash__(self):
        return hash((self.x, self.y))
a = Vec(1, 2)
b = Vec(1, 2)
print('a == b:', a == b)
print('len({a, b}):', len({a, b}))
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 21. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
    def __hash__(self):
        return hash((self.x, self.y))
print('Vec(1, 2) == Vec(1.0, 2.0):', Vec(1, 2) == Vec(1.0, 2.0))
print('Vec(1, 2) == Vec(True, 2):', Vec(1, 2) == Vec(True, 2))
print('len({Vec(1, 2), Vec(1.0, 2.0), Vec(True, 2)}):', len({Vec(1, 2), Vec(1.0, 2.0), Vec(Tru
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 22. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
    def __hash__(self):
        return hash((self.x, self.y))
a = Vec(float('nan'), 2)
b = Vec(float('nan'), 2)
print('a == b:', a == b)
print('hash(a) == hash(b):', hash(a) == hash(b))
print('len({a, b}):', len({a, b}))
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 10 / 16 = 0.62

Note 23. The `@dataclass` decorator examines the class body, and generates `__init__`, `__repr__`, and `__eq__` methods based on the field annotations. With `frozen=True`, it also generates `__hash__`. With `eq=True, frozen=False`, it sets `__hash__ = None`, matching the behavior of hand-written `__eq__` from the previous section.

Problem 24. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
from dataclasses import dataclass
@dataclass
class Vec:
    x: int
    y: int
a = Vec(1, 2)
b = Vec(1, 2)
print('a =', a)
print('a == b:', a == b)
try:
    print('hash(a):', hash(a))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 25. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
from dataclasses import dataclass
@dataclass(frozen=True)
class Vec:
    x: int
    y: int
a = Vec(1, 2)
b = Vec(1, 2)
print('a == b:', a == b)
print('len({a, b}):', len({a, b}))
try:
    a.x = 5
    print('a.x =', a.x)
except Exception as e:
    print(type(e).__name__)
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 26. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
from dataclasses import dataclass
@dataclass
class VecA:
    x: int
    y: int
@dataclass(frozen=True)
class VecB:
    x: int
    y: int
try:
    hash(VecA(1, 2))
    print('hash(VecA) works')
except TypeError:
    print('TypeError for VecA')
try:
    hash(VecB(1, 2))
    print('hash(VecB) works')
except TypeError:
    print('TypeError for VecB')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 27. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
from dataclasses import dataclass, field
@dataclass
class Foo:
    xs: list = field(default_factory=list)
a = Foo()
b = Foo()
a.xs.append('A')
print('a.xs=', a.xs)
print('b.xs=', b.xs)
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 28. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
from dataclasses import dataclass
@dataclass
class Vec:
    x: int
    y: int
@dataclass
class Vec3(Vec):
    z: int
v = Vec3(1, 2, 3)
print('v =', v)
print('v.x =', v.x)
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Note 29. The `@dataclass` decorator generates `__init__`, `__repr__`, `__eq__`, and comparison methods, but it never generates arithmetic like `__add__` or `__mul__`. There is no `arith=True` option, because there is no obvious default for `+`: should `Vec(1, 2) + Vec(3, 4)` be componentwise, or dot product, or something else? Every library that defines arithmetic on its own type (`complex`, `datetime`, `numpy`, `torch`) hand-writes the arithmetic dunders.

Problem 30. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
from dataclasses import dataclass
@dataclass
class Vec:
    x: int
    y: int
try:
    print('Vec(1, 2) + Vec(3, 4) =', Vec(1, 2) + Vec(3, 4))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 31. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
from dataclasses import dataclass
@dataclass
class Vec:
    x: int
    y: int
    def __add__(self, other):
        if isinstance(other, Vec):
            return Vec(self.x + other.x, self.y + other.y)
        return Vec(self.x + other, self.y + other)
print('Vec(1, 2) + Vec(3, 4) =', Vec(1, 2) + Vec(3, 4))
print('Vec(1, 2) + 10 =', Vec(1, 2) + 10)
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Note 32. When python evaluates `a + b`, it first tries `a.__add__(b)`. If that returns `NotImplemented` (or if `a` does not define `__add__`), python tries the *reflected* operation `b.__radd__(a)`. This is what allows `2 * Vec(1, 2)` to work even though `int` does not know how to multiply itself by a `Vec`.

Problem 33. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
from dataclasses import dataclass
@dataclass
class Vec:
    x: int
    y: int
    def __mul__(self, other):
        return Vec(self.x * other, self.y * other)
print('Vec(1, 2) * 3 =', Vec(1, 2) * 3)
try:
    print('3 * Vec(1, 2) =', 3 * Vec(1, 2))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 34. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
from dataclasses import dataclass
@dataclass
class Vec:
    x: int
    y: int
    def __add__(self, other):
        return Vec(self.x + other.x, self.y + other.y)
xs = [Vec(1, 2), Vec(3, 4), Vec(5, 6)]
try:
    print('sum(xs) =', sum(xs))
except TypeError:
    print('TypeError')
print('sum(xs, Vec(0, 0)) =', sum(xs, Vec(0, 0)))
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Note 35. For `a += b`, python will use `a.__iadd__(b)` if it exists, and otherwise fall back to `a = a + b`. Mutable containers like `list` define `__iadd__` (which extends the list in place), while immutable types like `tuple` and `str` do not. The difference is observable through aliasing: if you write `ys = xs` and then `xs += [1]`, with a list, `ys` also sees the change, but with a tuple, `ys` still refers to the old object.

Problem 36. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [1, 2, 3]
ys = xs
xs += [4]
print('xs=', xs)
print('ys=', ys)
zs = (1, 2, 3)
ws = zs
zs += (4,)
print('zs=', zs)
print('ws=', ws)
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 37. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
from dataclasses import dataclass
@dataclass
class Vec:
    x: int
    y: int
    def __add__(self, other):
        return Vec(self.x + other.x, self.y + other.y)
    def __iadd__(self, other):
        self.x += other.x
        self.y += other.y
a = Vec(1, 2)
a += Vec(3, 4)
print('a =', a)
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 12 / 16 = 0.75

Note 38. The `@total_ordering` class decorator is applied to a class that has defined `__eq__` and one of the four comparison operators <, <=, >, >=. It generates the other three comparison operators from those. Like `@dataclass`, it is a class decorator: it takes a class, adds methods, and returns the (possibly modified) class.

Problem 39. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
from dataclasses import dataclass
@dataclass
class Vec:
    x: int
    y: int
    def __lt__(self, other):
        return (self.x, self.y) < (other.x, other.y)
a = Vec(1, 2)
b = Vec(3, 4)
print('a < b:', a < b)
try:
    print('a >= b:', a >= b)
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 40. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import functools
from dataclasses import dataclass
@functools.total_ordering
@dataclass
class Vec:
    x: int
    y: int
    def __lt__(self, other):
        return (self.x, self.y) < (other.x, other.y)
a = Vec(1, 2)
b = Vec(3, 4)
print('a < b:', a < b)
print('a >= b:', a >= b)
print('a <= b:', a <= b)
print('a > b:', a > b)
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Note 41. Not every ordering is a *total* order. For example, we could compare `Vec`s by Pareto domination: `a <= b` means every component of `a` is no larger than the corresponding component of `b`. Then `Vec(1, 2)` and `Vec(2, 1)` are incomparable: neither is less than or equal to the other. Python's `sorted` does not require a total order, only one that is consistent, and returns *some* linear extension of the partial order.

Problem 42. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __le__(self, other):
        return self.x <= other.x and self.y <= other.y
    def __repr__(self):
        return f'Vec({self.x}, {self.y})'
a = Vec(1, 2)
b = Vec(2, 1)
print('a <= b:', a <= b)
print('b <= a:', b <= a)
print('a <= a:', a <= a)
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 43. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __lt__(self, other):
        return self.x <= other.x and self.y <= other.y and (self.x, self.y) != (other.x, othe
    def __repr__(self):
        return f'Vec({self.x}, {self.y})'
xs = [Vec(3, 3), Vec(1, 1), Vec(2, 2)]
print('sorted(xs):', sorted(xs))
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Note 44. `NotImplemented` is a special value, distinct from the exception `NotImplementedError`. When a dunder method returns `NotImplemented`, python interprets that as "I do not know how to perform this operation" and tries the reflected operation on the other operand. For example, evaluating `1.0 == Vec(1.0, 2.0)` calls `float.__eq__(1.0, Vec(...))`, which returns `NotImplemented`, so python falls back to identity comparison and returns `False`.

Problem 45. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __eq__(self, other):
        if not isinstance(other, Vec):
            return NotImplemented
        return self.x == other.x and self.y == other.y
v = Vec(1.0, 2.0)
print('v == v:', v == v)
print('1.0 == v:', 1.0 == v)
print('v == 1.0:', v == 1.0)
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 46. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __lt__(self, other):
        if not isinstance(other, Vec):
            return NotImplemented
        return (self.x, self.y) < (other.x, other.y)
    def __gt__(self, other):
        if not isinstance(other, Vec):
            return NotImplemented
        return (self.x, self.y) > (other.x, other.y)
v = Vec(1, 2)
print('v > Vec(0, 0):', v > Vec(0, 0))
print('v < Vec(2, 2):', v < Vec(2, 2))
try:
    print('v > 5:', v > 5)
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Note 47. The decorators `@dataclass` and `@total_ordering` are just functions that take a class, add methods to it, and return the modified class. We can write our own class decorators in exactly the same way. This is how libraries like `attrs`, `torch.nn.Module`, and every ORM define their classes.

Problem 48. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def add_arithmetic(cls):
    def __add__(self, other):
        return cls(self.x + other.x, self.y + other.y)
    def __radd__(self, other):
        return self
    cls.__add__ = __add__
    cls.__radd__ = __radd__
    return cls

@add_arithmetic
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __repr__(self):
        return f'Vec({self.x}, {self.y})'

xs = [Vec(1, 2), Vec(3, 4), Vec(5, 6)]
print('sum(xs) =', sum(xs))
print('Vec(1, 2) + Vec(3, 4) =', Vec(1, 2) + Vec(3, 4))
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 49. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def algebraic(fields):
    def decorate(cls):
        def __add__(self, other):
            return cls(*[getattr(self, f) + getattr(other, f) for f in fields])
        cls.__add__ = __add__
        return cls
    return decorate

@algebraic(fields=('x', 'y'))
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __repr__(self):
        return f'Vec({self.x}, {self.y})'

print('Vec(1, 2) + Vec(3, 4) =', Vec(1, 2) + Vec(3, 4))
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

LLM Model Performance

